

Reviewer Pack — Minimal Order of Coxeter Group Conjugacy Classes and Communi...

Entry 763bb5edd256 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 02:48:04 UTC

Minimal Order of Coxeter Group Conjugacy Classes and Communication Complexity Rank Variance

Entry ID: 763bb5edd256 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 02:48:04 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory (Coxeter Groups) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given CNF formula ϕ with n variables, the variance in the rank of the associated matrix representations of ϕ over different Coxeter groups is bounded by a function $f(n) = \Theta(\log(n))$.

Rationale (proposer's reasoning):

Coxeter groups encode symmetries and geometric structures that could provide insights into the underlying complexity of communication tasks. If the conjecture holds, it suggests that the structure of Coxeter groups can be exploited to analyze communication complexity, potentially leading to new algorithms or lower bounds.

Taxonomy category: geometric_group_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 15701f22d8dae9a4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The variance in the rank of matrix representations of CNF formulas over Coxeter groups is bounded by $\Theta(\log(n))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group conjugacy classes" AND "communication complexity rank variance" - "matrix representation" AND ("Coxeter groups" OR "minimal order") AND communication complexity - "CNF formula" AND matrix rank AND Coxeter groups AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/1404.3510v1] Minimal Faithful Upper-Triangular Matrix Representations for Solvable Lie Algebras - [http://arxiv.org/abs/1206.1775v1] Exponential Time Complexity of the Permanent and the Tutte Polynomial - [http://arxiv.org/abs/1402.0054v1] Popular conjectures imply strong lower bounds for dynamic problems - [http://arxiv.org/abs/2408.03465v2] Algorithms for the Diverse-k-SAT problem: the geometry of satisfying assignments

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def matrix_representation(cnf):
        n = max(abs(x) for clause in cnf for x in clause)
        M = [[0] * (n + 1) for _ in range(n + 1)]
        var_indices = {var: i for i, var in enumerate(range(-n, 0), start=1)}

        for clause in cnf:
            for var in clause:
                if var < 0:
                    M[var_indices[var]][-var] += 1
                else:
                    M[-var][var_indices[var]] += 1
```

```

    return M

def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    augmented_matrix = [row + [0] for row in matrix]

    # Gaussian elimination
    for i in range(min(m, n)):
        if augmented_matrix[i][i] == 0:
            for j in range(i + 1, m):
                if augmented_matrix[j][i] != 0:
                    augmented_matrix[i], augmented_matrix[j] = augmented_matrix[j], augmented_matrix[i]
                    break
            else:
                continue

        pivot = augmented_matrix[i][i]
        for j in range(n + 1):
            augmented_matrix[i][j] /= pivot

        for j in range(m):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(n + 1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    rank = sum(1 for row in augmented_matrix if any(row))
    return rank

n_values = [5, 10, 15, 20, 30, 40]
ranks = []

for n in n_values:
    cnf = generate_cnf(n)
    M = matrix_representation(cnf)
    rank_value = rank(M)
    ranks.append(rank_value)

mean_rank = sum(ranks) / len(ranks)
variance_rank = sum((x - mean_rank) ** 2 for x in ranks) / len(ranks)

f_n = math.log(n_values[-1])
if variance_rank > f_n:
    conjecture_holds = False
    counterexample = "variance exceeds f(n)"
else:
    conjecture_holds = True
    counterexample = ""

return {
    "metric_name": "Variance of Rank",
    "metric_value": variance_rank,
    "instances_tested": len(ranks),
    "n_max": n_values[-1],
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_variance = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_variance} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_variance} std=0.0 support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = result["counterexample"]
                first_failing_seed = seed
                break

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c056c03f.py", line 106, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c056c03f.py", line 76, in run_trial
    M = matrix_representation(cnf)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c056c03f.py", line 40, in matrix_representation
    M[-var][var_indices[var]] += 1
    ~~~~~^
KeyError: 5

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that neither the support nor falsification conditions could be evaluated. | next: Re-run the test to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16262
2	preregistration	ollama_remote	glm4:latest	0	0	9352
3	novelty	ollama_remote	glm4:latest	0	0	8944
4	novelty	ollama_remote	glm4:latest	0	0	9810
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19061
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12836
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11172
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11343
9	judge	ollama_remote	glm4:latest	0	0	21805

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120585 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/763bb5edd256.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/763bb5edd256.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/763bb5edd256.tar.gz (if generated)